

EN UNA FRASE Rechazas los datos inválidos en la capa de entrada con Bean Validation y `@Valid`, y centralizas los errores en un `@RestControllerAdvice` que responde con `ProblemDetail` (RFC 9457): 400 con el detalle de cada campo y 404 cuando el libro no existe.

LO ESENCIAL

<code>spring-boot-starter-validation</code>	Starter que trae Hibernate Validator; no viene incluido en <code>spring-boot-starter-web</code>
<code>@NotBlank</code>	El <code>String</code> no puede ser <code>null</code> , vacío ni solo espacios (<code>@NotEmpty</code> sí admite espacios)
<code>@NotNull</code>	El campo no puede ser <code>null</code> ; funciona con cualquier tipo
<code>@Min / @Max</code>	Valor mínimo y máximo (inclusivos) para campos numéricos
<code>@Size(min, max)</code>	Longitud mínima y máxima para <code>String</code> y colecciones
Integer en vez de int	Con <code>int</code> , Jackson pone 0 si el campo no llega; con <code>Integer</code> queda <code>null</code> y <code>@NotNull</code> lo detecta
<code>@Valid</code>	Delante del parámetro <code>@RequestBody</code> : activa la validación antes de ejecutar el método
<code>MethodArgumentNotValidException</code>	La lanza Spring cuando falla la validación de un <code>@RequestBody</code>
<code>@RestControllerAdvice</code>	Clase que maneja las excepciones de todos los controladores
<code>@ExceptionHandler(X.class)</code>	Método que maneja un tipo de excepción concreto
<code>ProblemDetail</code>	Formato de error RFC 9457: <code>type</code> , <code>title</code> , <code>status</code> , <code>detail</code> , <code>instance</code> ; extras con <code>setProperty(clave, valor)</code>
<code>LibroNoEncontradoException</code>	Excepción de dominio propia (<code>RuntimeException</code>) que el manejador traduce a 404
<code>@ResponseStatus(HttpStatus.CREATED)</code>	Fija el código de estado cuando el método ya no devuelve <code>ResponseEntity</code>
Errores 4xx / 5xx	4xx: el fallo es del cliente (400, 404, 409); 5xx: el fallo es del servidor (500, 503)

ERRORES TÍPICOS

1 Un libro inválido se guarda con 201 y sin ningún error

✓ `@Valid` está sobre el método; va delante del parámetro: `crear(@Valid @RequestBody Libro libro)`.

2 package jakarta.validation.constraints does not exist al compilar

✓ falta `spring-boot-starter-validation` en el `pom.xml`.

3 UnexpectedTypeException: No validator could be found for constraint 'NotBlank' validating type 'int'

✓ `@NotBlank` solo vale para `String`; en números usa `@NotNull` (con `Integer`) junto con `@Min` y `@Max`.

4 El manejador nunca se ejecuta y llega el 400 genérico de Spring

✓ capturas `ValidationException`; la que lanza `@Valid` es `MethodArgumentNotValidException`.

5 IllegalStateException: Ambiguous @ExceptionHandler method mapped al arrancar

✓ dos `@RestControllerAdvice` manejan la misma excepción; reúne todos los `@ExceptionHandler` en una única clase.

EN EL PROYECTO BIBLIOTECA

Entra `spring-boot-starter-validation` en el `pom.xml` y `Libro` se anota con restricciones (anio pasa de `int` a `Integer`). Nacen `dominio/LibroNoEncontradoException` y `controlador/ManejadorGlobalErrores`. `BibliotecaService` gana `buscarPorIdOFallar`, y `actualizar/eliminar` lanzan la excepción en vez de devolver `Optional/boolean`; `LibroController` usa `@Valid` y `@ResponseStatus` y deja de usar `ResponseEntity`. `pruebas.http` no cambia en este capítulo.

■ CÓDIGO MÍNIMO Y PETICIONES

Las restricciones en `dominio/Libro.java` (`autor` se anota igual que `titulo`, con máximo 150).

```
@NotBlank(message = "El título es obligatorio")
@Size(min = 1, max = 200, message = "El título no puede superar los 200 caracteres")
private String titulo;

@NotNull(message = "El año es obligatorio")
@Min(value = 1450, message = "El año no puede ser anterior a 1450")
@Max(value = 2100, message = "El año no puede ser posterior a 2100")
private Integer anio;
```

JAVA

El manejador global en `controlador/ManejadorGlobalErrores.java`.

```
@RestControllerAdvice
public class ManejadorGlobalErrores {

    @ExceptionHandler(LibroNoEncontradoException.class)
    public ProblemDetail manejarLibroNoEncontrado(LibroNoEncontradoException ex) {
        ProblemDetail problema = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
        problema.setTitle("Libro no encontrado");
        return problema;
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ProblemDetail manejarValidacion(MethodArgumentNotValidException ex) {
        String detalle = ex.getBindingResult().getFieldErrors().stream()
            .map(error -> error.getField() + ": " + error.getDefaultMessage())
            .collect(Collectors.joining("; "));
        ProblemDetail problema = ProblemDetail.forStatusAndDetail(HttpStatus.BAD_REQUEST, detalle);
        problema.setTitle("Datos de entrada no válidos");
        return problema;
    }
}
```

JAVA

El controlador, ya sin `ResponseEntity` ni `Optional`: el servicio lanza la excepción y el manejador responde.

```
@GetMapping("/{id}")
public Libro obtenerPorId(@PathVariable Long id) {
    return service.buscarPorIdOFallar(id);
}

@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public Libro crear(@Valid @RequestBody Libro libro) {
    return service.registrar(libro);
}

// BibliotecaService
public Libro buscarPorIdOFallar(Long id) {
    return buscarPorId(id).orElseThrow(() -> new LibroNoEncontradoException(id));
}
```

JAVA